

Shell Scripting Cheat Sheet

❖ Create a summary table like this at the top of your cheat sheet:

Topic	Key syntax	Example
Shebang	<code>#!/bin/bash</code>	<code>#!/bin/bash</code>
Running scripts	<code>Chmod +x</code>	<code>Chmod +x 764 file.txt</code>
Comments, Inline	<code>#code for condition3....</code>	<code>#code for condition3....</code>
VAR	<code>VAR=" HELLO"</code>	<code>NAME="Devops"</code>
argument	<code>\$1 \$2</code>	<code>./script.sh arg1</code>
if	<code>if [condition]; then</code>	<code>if [-file]; then</code>
for loop	<code>for i in list ; do</code>	<code>for i in 1 2 3; do</code>
function	<code>name() {.....}</code>	<code>greet() {echo "hi"};</code>
grep	<code>grep pattern file</code>	<code>grep -i "error" log.txt</code>
awk	<code>awk "{print \$1}" file</code>	<code>awk -f '{print \$1}' /etc/passwd</code>
sed	<code>sed 's/old/new/g' file</code>	<code>sed -i 's/foo/bar/g' config.txt</code>

Task 1: Basics

❖ Document the following with short descriptions and example:

- Shebang (`#!/bin/bash`) what it does and why it matters
 - The Shebang is an interpreter directive used in UNIX like operating systems. It is a character sequence, `#!` That constitutes the first line of the script.
 - This line specifies the absolute path to the system interpreter (such as `bash`, `python3`) required to execute the commands within the scripts.
- Running a script – `chmod +x`, `./script.sh`, `bash script.sh`
 - A Shell script is a text file containing a series of commands written for a shell, such as `bash`, `zsh`, `Sh`, to execute.
 - It automates repetitive tasks and simplifies complex operations. Can include programming constructs like loops, conditionals, and variables.
 - `Chmod +x` command is used to give permission to the files.

- `./script.sh` command that is used to run the executable script file.
- Bash `script.sh` command is used to run the file on the bash interpreter.
- Comments – single line (`#`) and inline
 - Single line comments are used to explain a single command or small block of code. They begin with `#` symbol, and everything after it on the same lines is ignored by the shell.
 - Explain the purpose of a command
 - Temporarily disable a command
 - Inline comment improves script readability
 - Add short notes for future references.
- Variables – declaring, using and quoting (`$VAR`, `"$VAR"`, `'$VAR'`)
 - There are two types of variables in shell scripting
 - Local variables
 - Environment variables
 - How to define variables No spaces: The most critical rule. `VAR=" HELLO"` is the correct. `VAR = "HELLO"` is a fatal error.
 - Variable data could be accessed by appending the variable name with `'$'` (`VAR_1="Devil"` `echo "$VAR_1"`)
 - `'$VAR'` prints literal text
- Reading user input – `read`
 - Changing the Delimiter: Using this argument we can change the way we assign the variables the value i.e. if we want to get multiple inputs using a single `read` command, we can do that using space separated values.
 - `read -p "Enter the name age and country:", name age country`
- Command-line arguments - `$0`, `$1`, `$#`, `$@`, `$?`

- Command-line arguments are values passed to a script when it is executed. These arguments can be accessed inside the script and used to control its behavior dynamically.
- Using \$#: Represents the total number of command line arguments.
- Using \$@: Represents all command line arguments as a list.
- Using \$0: The name of the script itself
- Using \$1: The indexing of the arguments starts at one, and the first argument can be accessed inside the script using \$1.

❖ Task 2: Operators and Conditionals

❖ Document with examples:

➤ String comparisons - =, !=, -z, -n

- “==” operator: Double equal to operator compares the two operands. It returns true if they are equal otherwise returns false.
- “!=” Operators: Not Equal to operator return true if the two operands are not equal otherwise it returns false.
- -n string: The length of string is nonzero
- -z string: The length of string is zero

➤ Integer comparisons = -eq, -ne, -lt, -gt, -le, -ge

- -eq: Equal to ([“\$a” -eq “\$b”])
- -ne: Not equal to ([“\$a” -ne “\$b”])
- -lt: Less than ([“\$a” -lt “\$b”])
- -gt: Greater than ([“\$a” -gt “\$b”])
- -le: Less than or equal to ([“\$a” -le “\$b”])
- -ge: Greater than or equal to ([“\$a” -ge “\$b”])

➤ File test operators - -f, -d, -e, -r, -w, -x, -s

- -f: File test operator in shell scripting checks if a file exists and is a regular file (not a directory)

- -d: This operator checks if the given directory exists or not. If it exists then operators return true otherwise false.
- -e: This operator checks whether the given file exists or not. If it exists or not. If it exists, this operator returns true otherwise false.
- -r: This operator checks whether the given file has read access or not. If it has read access, then it returns true otherwise false
- -w: This operator checks whether the given file has written access or not. If it has written, then it returns true otherwise false.
- -x: This operator checks whether the given file has execute access or not. If it has execute access, then it returns true otherwise false
- -s: This operator checks the size of the given file. If the size of the given file is greater than 0, then it returns true otherwise it is false.

➤ if , elif , else syntax:

- if [condition1]; then
 #code for condition...
- elif [condition2]; then
 #code for condition2....
- elif [condition3]; then
 #code for condition3....
- else
 # code to run if nothing matches...
- fi

➤ Logical operators – &&, ||, |

- Logical AND (&&): This is a binary operator, which returns true if both the operands are true otherwise returns false.
- Logical OR (||): This is a binary operator, which returns true if either of the operands is true or if both the operands are true. It returns false only if both operands are false.
- Bitwise OR (|): Bitwise | operator performs binary OR operation bit by bit on the operands.

➤ Case Statements – case esac

- A case statement in bash is used to execute different code blocks based on different conditions. It is cleaner than multiple if else statements when comparing one variable against many values
- case \$variable in

```
apple)
    echo "red or green "
;;
*)
    echo "unknown fruit"
;;
```

```
esac
```

❖ Task 3: Loops

❖ Document with examples:

➤ For loop –list-based and C –style

- iterating over a list of items

```
for a in 1 2 3 4 5 6 7 8 9 10
```

```
do
```

```
# if a = 5 then continue the loop and don't move to line 8
```

```
if [ $a == 5 ]; then
```

```
    continue
```

```
fi
```

```
echo "iteration no $a"
```

```
done
```

➤ While loop

```
a=0
```

```
# -gt is greater than operator
```

```
# iterate the loop until a is less than 10
```

```
# iterate is less than operator
```

```
while [ "$a" -lt 10 ]
```

```
do
```

```
    #print the values
```

```
    echo "$a"
```

```
    # increment the values
```

```
    a=$((a + 1))
```

```
done
```

➤ Until loop

```
▪ #!/bin/bash
```

```
A=0
```

```
# -gt is greater than operator
```

```
# Iterate the loop until a is greater than 10
```

```
Until [ $a -gt 10 ]
```

```
Do
```

```
    #print the values
```

```
    Echo $a
```

```
    # Increment the value
```

```
    A='expr $a + 1'
```

```
done
```

➤ Loop control –break, continue

```
▪ for i in range(8):
```

```

if i == 2:

    continue # Skip the rest of the loop body when i is 2

if i == 4:

    break    # Exit the loop completely when i is 4
print(i)

```

➤ Looping over files –for file in *.log

```

    for file in *.log

do

    if [ -f "$file" ]; then

        echo "File: $file"

        wc -l "$file" # Count lines in each file

    fi

done

```

➤ Looping over command output –while read line

```

    cat file.txt | while read line

do

    echo "Processing: $line"

done

```

❖ Task 4: Functions

❖ Document with examples:

➤ Defining a function –function_name() {.....}

```

    # Defining a function
    greet() {
        echo "hello, $1!"
    }

```

```
# calling a function
greet "lenin"
```

➤ Calling a function

- # Defining a function

```
greet() {
    echo "hello, $1!"
}
```

```
# calling a function
greet "lenin"
```

➤ Passing arguments to functions - \$1, \$2 inside functions

- add_numbers() {

```
    local sum=$(( $1 + $2 ))
```

```
    echo "Sum: $sum"
```

```
}
```

```
add_numbers 5 10
```

➤ Return values –return vs echo

- Using return (returns exit status 0-255)

```
check_file() {
    if [ -f "$1" ]; then return 0
```

```
# Success
```

```
else return 1 # Failure
```

```
fi
```

```
}
```

```
#Using echo (returns actual data)
```

```
get_filename() {
```

```
    echo "logfile.txt"
```

```
}
```

```
#Calling and checking return value
```

```
if check_file "test.log"; then
```

```
echo "File exists"
```



```

fi
#Calling and capturing echo output
filename=$(get_filename)

echo "Got: $filename"

```

➤ Local variables –local

```

▪ process_data() {
    local name="$1"      # Local variable
    global_var="global"  # Global variable (no 'local' keyword)

    echo "Name: $name"
    echo "Global: $global_var"
}
process_data "John"
echo "Outside function: $global_var" # Accessible
echo "Outside function: $name"      # Empty (local scope)

```

❖ Task 5: Text Processing commands

➤ Grep – search patterns in each file. Patterns are one or more patterns separated by newline characters, and grep prints for each line that matches a pattern. Typically, patterns should be quoted when grep is used in a shell command.

- -i: Ignore case distinctions in patterns and input data, so that characters that differ only in case match each other.
- -r: Read all files under each directory, recursively, following symbolic links only if they are on the command line.
- -c: Count suppress normal output; instead print a count of matching lines for each input file.
- -n: Line-number prefix each line of output with the 1 based line number within its inputs file.
- -v: Invert the sense of matching, select non-matching lines.
- -E: Interpret patterns as extended regular expression

➤ Awk –print columns, field separator, patterns, BEGIN/END

- Awk '{print \$1, \$3}' file.txt
- Awk -F':' '{print \$1}' /etc/passwd
- Awk '/error/ {print \$0}' log.txt
- Awk 'BEGIN {print "Start"} {sum += \$1} END {print "Total:", sum}' file.txt
- Sed –substitution, delete lines, in-place edit
 - Sed 's/foo/bar' file.txt
 - Sed 's/foo/bar/g' file.txt
 - Sed '/^#/d' file.txt
 - Sed -i 's/old/new/g' file.txt
- Cut –extract columns by delimiter
 - Cut -d':' -f1 /etc/passwd
- Sort –alphabetical, numerical, reverse, unique
 - Sort file.txt # alphabetical
 - Sort -n file.txt # numerical
 - Sort -r file.txt reverse
 - Sort -u file.txt #unique
 - Sort -t ':' -k3 -n /etc/passwd # by field
- Uniq –deduplicate, count
 - sort file.txt | uniq #Remove duplicates (input must be sorted)
 - sort file.txt | uniq -c #Count occurrences
 - sort file.txt | uniq -d #Show only duplicates
- Tr –translate/delete characters
 - echo "hello" | tr 'a-z' 'A-Z' # Lowercase to uppercase
 - echo "h3ll0" | tr -d '0-9' # Delete characters
- Wc –line/word/char count
 - wc -l file.txt # Line count
 - wc -w file.txt # Word count
 - wc -c file.txt # Byte/char count
- Head/ tail –first/last N lines, follow mode

- `head -n 5 file.txt` # First 5 lines
- `tail -n 5 file.txt` # Last 5 lines

❖ Task 6: Useful patterns and one-liners

- Include at least 5 real-world you find useful. Examples:
- Find and delete files older than N days
 - `find /path/to/directory -type f -mtime +30 -delete`
- Count lines in all .log files
 - `wc -l *.log | tail -1`
- Replace a string across multiple files
 - `sed -i 's/old_string/new_string/g' *.txt`
- Check if a service is running
 - `systemctl is-active --quiet nginx`
- Tail a log and filter for errors in real time
 - `tail -f app.log | grep --line-buffered ERROR`

❖ Task 7: Error Handling and debugging

- Document with examples:
- Exit codes -- \$?, exit 0, exit 1
 - `#!/bin/bash`
 - `# run a command`
 - `ls /nonexistent 2>/dev/null`
 - `# check the exit code`
 - `if [ $? -eq 0 ]; then`
 - `echo "success"`
 - `else`
 - `echo "failed with exit code: $?"`
 - `fi`
 - `# explicit exit codes`
 - `if [ -f "file.txt" ]; then`
 - `echo "file exists"`
 - `exit 0`
 - `else`
 - `echo "file not found"`

- exit 1
- fi
- Set -e -exit on error
 - #!/bin/bash
 - set -e
 - echo "step 1"
 - cd /tmp
 - echo "step 2"
 - ls /nonexistent # script stops here, doesn't continue
 - echo "step 3" # never reaches here
- Set -u -treat unset variables as error
 - #!/bin/bash
 - set -u
 - name="john"
 - echo "name: \$name"
 - echo "age: \$age" # error! \$age is unset, script stops
- Set -o pipefail -catch errors in pipes
 - #!/bin/bash
 - set -o pipefail
 - # without pipefail, this succeeds (grep returns 0)
 - cat /nonexistent | grep "text"
 - echo "exit code: \$?" # would show 0
 - # with pipefail, it fails (cat returns 1)
 - # exit code: 1
- Trap -trap 'cleanup' EXIT
 - #!/bin/bash
 - set -e
 - set -u
 - set -o pipefail
 - # define cleanup function
 - cleanup() {
 - echo "cleaning up..."

- `rm -f /tmp/tempfile_$$`
- `echo "done!"`
- `}`
- `# set trap to run cleanup on exit (success or error)`
- `trap 'cleanup' exit`
- `echo "creating temp file..."`
- `touch /tmp/tempfile_$$`
- `echo "working with temp file..."`
- `# cleanup runs automatically when script ends`